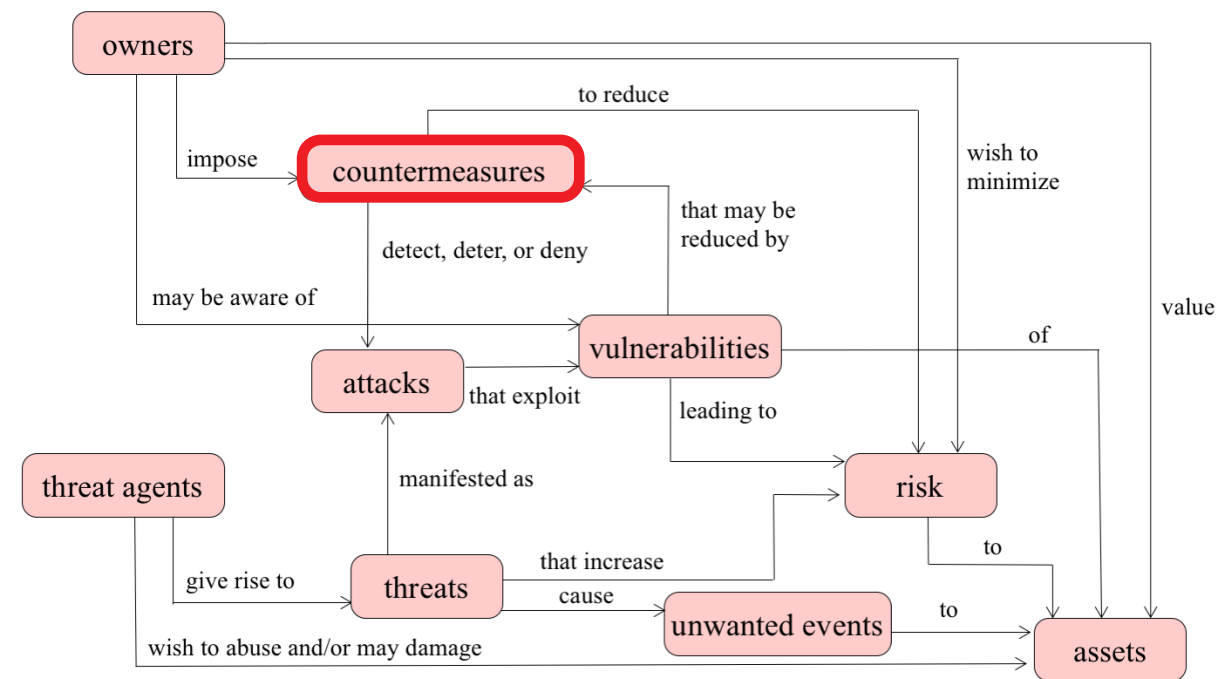
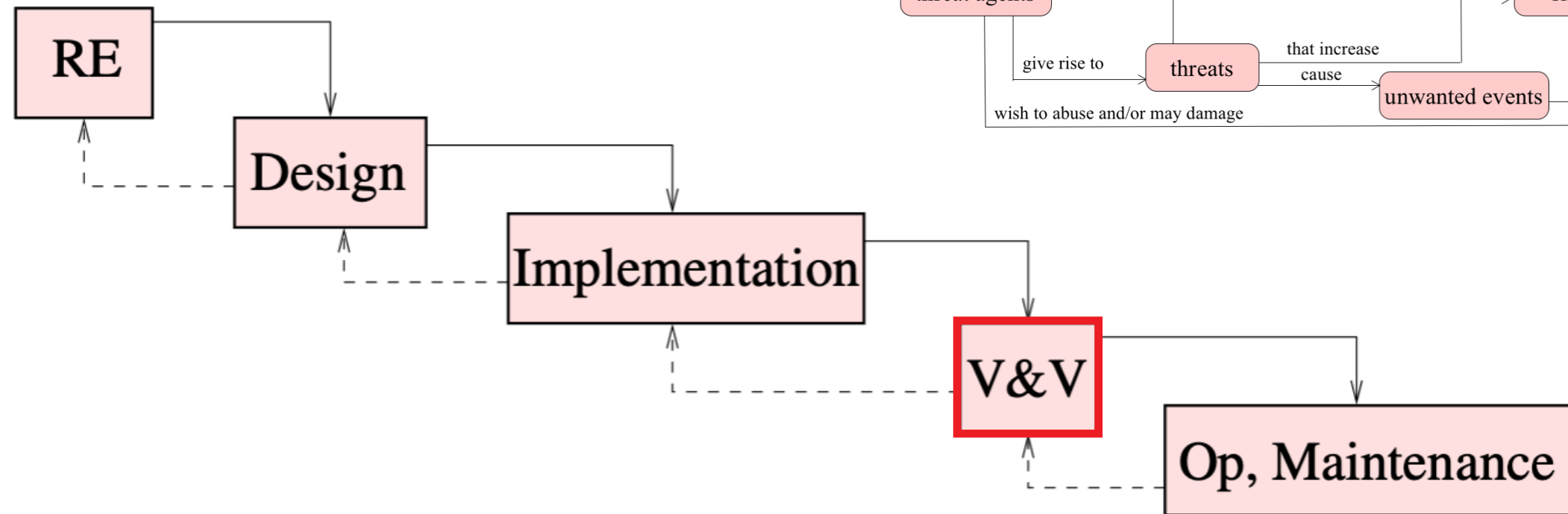


Code Scanning

Security Engineering
Srđan Krstić
ETH Zurich

Where are we?



- Code scanning is a verification technique
- Takes code as input
- Supplements manual code inspection
- Countermeasure

Road map

- Motivation
- Code scanning and security
- What is code scanning?
- Code scanning: Theory and practice
- Bug-specific algorithms
- Conclusions and practical experience

Motivation and problem context

What are best practice?

Critical system example



- Mars curiosity rover: 3.8M lines of code
- **Critical**: small mistakes means loss of mission
 - Moreover, missions are increasingly complex
- NASA takes this extremely seriously
 - Hired many top researchers
 - Extensive Quality Assurance program
- Central part of Quality Assurance: **Risk-Based Coding Standards**
 - 6 Levels of Compliance (LOC)
 - More critical $\Rightarrow$ higher level

NASA Levels of Compliance

- **LOC 1 (2 rules):** Language compliance, C-language standard ISO-C99
 - So code cannot rely on any compiler extensions
 - No compiler warnings & scanner warnings allowed
 - `gcc -Wall -std=c99 -pedantic-errors`
- **LOC 2 (10 rules):** Predictable execution
 - **Example:** all loops have statically verifiable upper bounds on number of iterations possible
- **LOC 3 (7 rules):** Defensive coding
 - Assertion density in code must be higher than 2%
 - Assertions used for testing and remain enabled during missions
 - Failing assertions tied with fault-protection system whereby spacecraft goes into a predefined safe state

NASA Levels of Compliance (cont.)

- **LOC 4 (12 rules):** Code clarity
 - target level for mission-critical code (all on-board software)
 - Restrictions to CPP, function pointers, and pointer indirection
 - 31 coding rules by this point
- **LOC 5 & 6:** Safety-critical requirements (shall & should rules)
 - All rules from MISRA C coding guidelines (not covered previously)
MISRA = Motor Industry Software Reliability Association
 - 143 rules (checkable using static program analysis)
+ 16 "directives" concerning procedural manners.
 - **Example:** do not use malloc, as it may fail
 - All MISRA rules checkable by *Coverity* and *CodeSonar*

Code scanning at NASA

- Multiple code scanners used: *Coverity*, *CodeSonar*, *Semmlle*, and *Uno*
 - All reported flaws are logged
- Supplement manual code reviews. Flaws also logged.
- Developers must respond to, or remediate, flaws found by both!
- Some statistics
 - 2008-2012: 10,000 peer comments, 30,000 scanner comments
 - Ca. 84% led to code changes
 - Ca. 12% end up in explicit disagree responses

NASA conclusions

- Extremely positive experience with code scanning
“A static analyzer will not tire of checking for the same types of defects over and over, night after night, patiently reporting all violations”
- They also use model-checkers (*Spin*)
 - Used for selected critical components, e.g. with tricky concurrency
 - Automatically extract models from C source-code in some cases
 - Manually constructed models in others
- Excellent results: system reliability (now) very high.

Code analysis and security

A trivial example

- C function to print a message to a given file descriptor

```
void printMsg(FILE* file, char* msg) {  
    fprintf(file, msg);  
}
```

- Defensive version

```
void printMsg(FILE* file, char* msg) {  
    if (file == NULL) {  
        logError("attempt to print message to null file");  
    } else if (msg == NULL) {  
        logError("attempt to print null message");  
    } else {  
        fprintf(file, msg);  
    }  
}
```

- Is this now OK?

What can an adversary do?

- Could enter: AAA1_%08x.%08x.%08x.%08x.%08x.%n
 - This is a “format string” exploit, analogous to a buffer overflow.
 - Input seems absurd. But this doesn’t stop the adversary
 - Inputs from untrusted sources can always be **tainted**
- Solution here

```
void printMsg(FILE* file, char* msg) {
    if (file == NULL) {
        logError("attempt to print message to null file");
    } else if (msg == NULL) {
        logError("attempt to print null message");
    } else {
        fprintf(file, "%.128s", msg);
    }
}
```
- Programmers easily make such errors
Code scanners should help find them!

Scope: static analysis applied to implementation code not design

	Visible in the code	Visible only in the design
Generic defects	Static analysis sweet spot. Built-in rules make it easy for tools to find these without programmer guidance. • <i>Example: buffer overflow.</i>	Most likely to be found through architectural analysis. • <i>Example: the program executes code downloaded as an email attachment.</i>
Context-specific defects	Possible to find with static analysis, but customization may be required. • <i>Example: mishandling of credit card information.</i>	Requires both understanding of general security principles along with domain-specific expertise. • <i>Example: cryptographic keys kept in use for an unsafe duration.</i>

Problem categories (from security perspective)

1. **Input validation and representation**
 - Buffer overflows, XSS, injection attacks, etc.
2. **API abuse**: abuse contract between caller and callee
 - Provide wrong input or make too strong assumptions about output
3. **Security features**: e.g., authorization engine like Cedar
 - $\neq$ Secure features, e.g., endpoint implementation does not call Cedar
4. **Time and state**: e.g., race condition example
5. **Error handling**: handling errors poorly or not at all
6. **Code quality**: dereference null pointers, infinite loops, etc.
7. **Encapsulation**: lack of it
- *. **Environment**: problems outside of the source code

What is code scanning?

Definition and examples

Scope

- **Rough definition: code scanning = pragmatic static analysis**
Analyze source code only. No testing, runtime verification, etc.
- As we will see, **pragmatism is essential**
- Based on research community successes
 - Type checking
 - Property checking (model-checking)
 - Abstract interpretation
- Also on software engineering best practices
 - Style checking, program comprehension, security reviews, etc.

Let's consider a few examples

Type checking

```
short s = 0;  
int i = s;  
short r = i;
```

- The Java compiler will flag this as an error. Right?

```
Object[] objs = new String[1];  
objs[0] = new Object();
```

- How about this?
What will happen at runtime?
- Type checkers are useful
 - But may suffer from false positives/negatives
 - Identifying which computations are harmful is **undecidable**

Style checking

- Enforce more picker and more superficial rules than type checkers
- Some compilers can check these
E.g., gcc -Wall checks cases

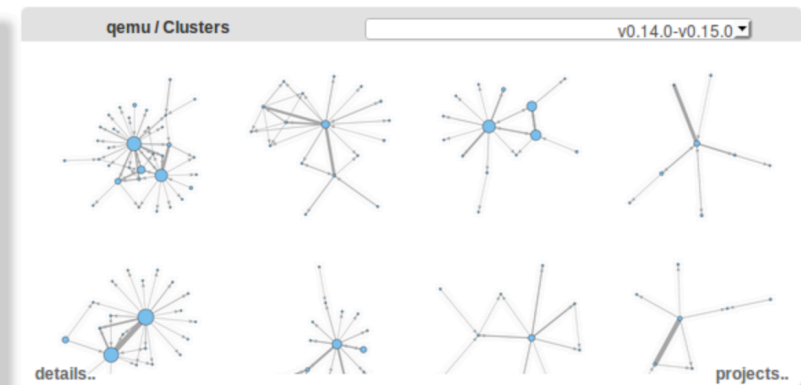
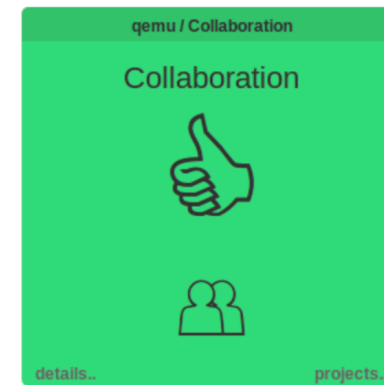
```
1 typedef enum { red, green, blue } Color;
2
3 char* getColorString(Color c) {
4     char* ret = NULL;
5     switch (c) {
6         case red:
7             printf("red");
8     }
9     return ret;
10 }
```

```
enum.c:5: warning: enumeration value 'green' not handled in switch
enum.c:5: warning: enumeration value 'blue' not handled in switch
```

- Style checkers often extensible
PMD for Java is a good example of this

Program understanding

- Tools can help with
 - Making sense of large code base
 - Reverse engineering abstractions
 - Code slicing
 - Finding declarations and uses
- All this is useful for manual code reviews

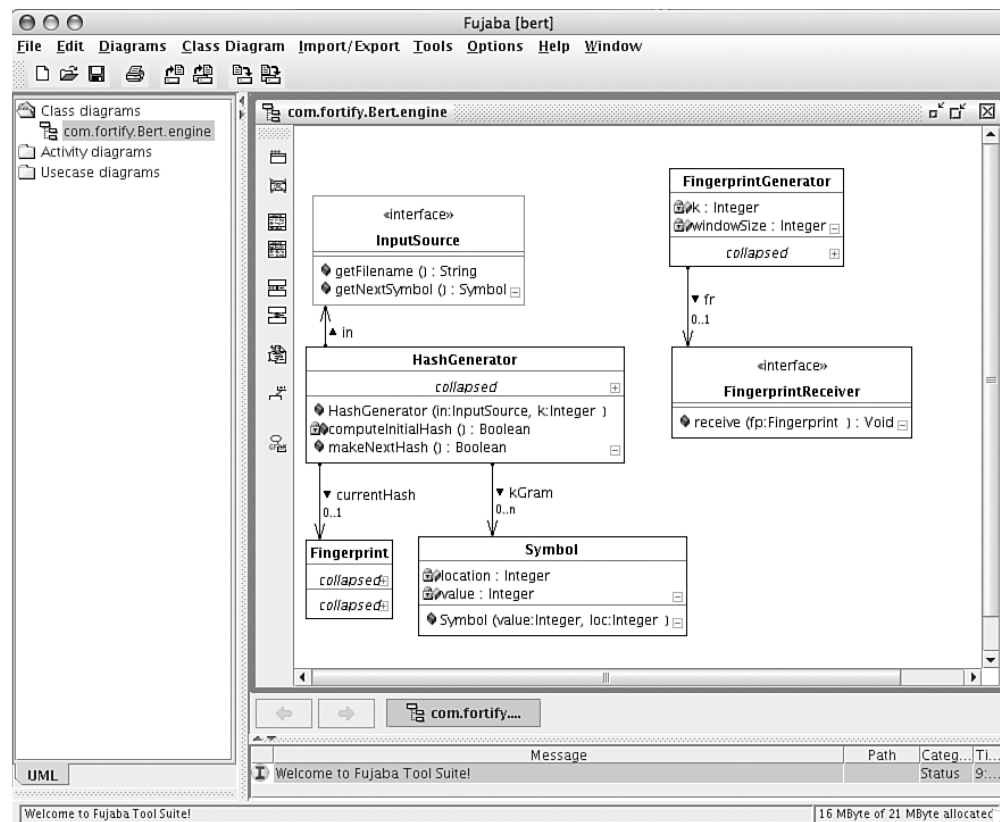
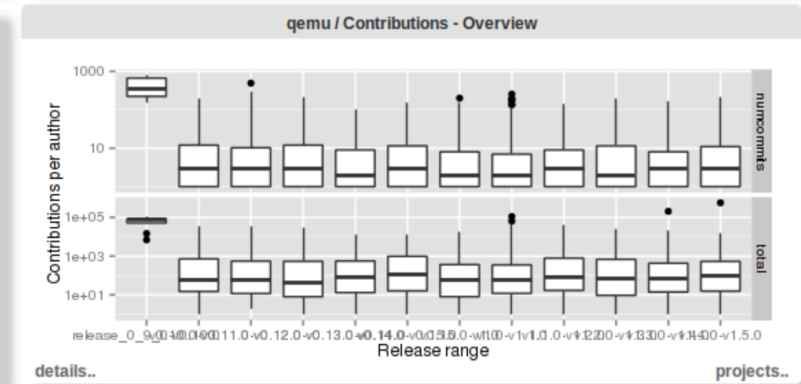


qemu / Commits

v0.14.0-v0.15.0

	Name	Number of commits
1	Peter Maydell	148.00
2	Jan Kiszka	132.00
3	Daniel Kvasnicka	104.00
4	Blue Swirl	73.00
5	Alexander Graf	68.00
6	Aurelien Jarno	65.00

details... projects...



Search Results - Search Method by Coupling

Method by Coupling

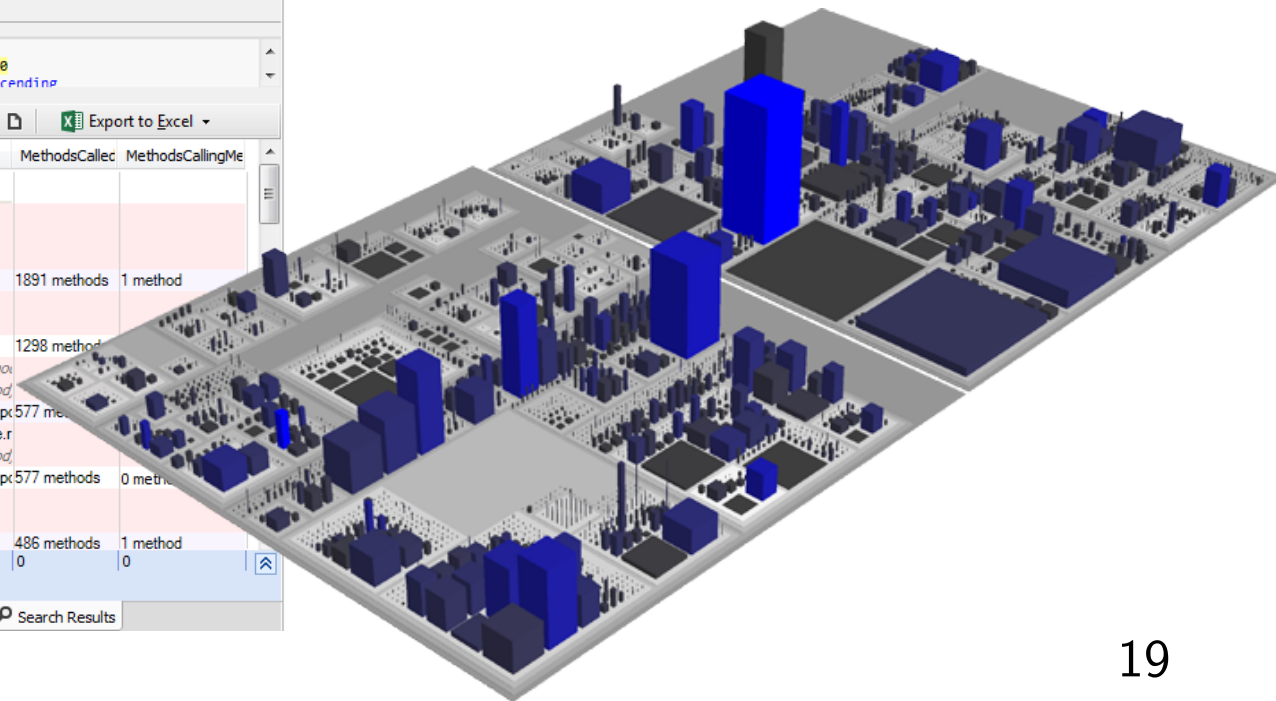
Search methods that are using more than 210 methods

Code Query Generated

```
from m in Application.Methods
where m.NbMethodsCalled >= 210
orderby m.NbMethodsCalled descending
```

methods	MethodsCalled	MethodsCallingMe
rt (36 methods)		
javax.swing.plaf.nimbus (1 method)		
NimbusDefaults (1 method)		
initializeDefaults(UIDefaults)	1891 methods	1 method
sun.awt (1 method)		
ExtendedKeyCodes (1 method)		
ExtendedKeyCodes()	1298 methods	
javax.management.remote.rmi (1 method)		
_RMIConnectionImpl_Tie (1 method)		
_invoke(String,InputStream,Response)	577 methods	
org.omg.stub.javax.management.remote.rmi (1 method)		
_RMIConnectionImpl_Tie (1 method)		
_invoke(String,InputStream,Response)	577 methods	
sun.io (1 method)		
CharacterEncoding (1 method)		
installAll()	486 methods	1 method
Sum	0	0

Class Browser Queries and Rules Edit Search Results



Property checking (model-checkers, etc.)

- Check all behaviors against property specifications.
- Consider following code:
 - What property might you require?
 - How could you check this?
- Model-checkers are powerful. But:
 - State space explosion is a problem
 - You must specify properties and often the model too
- Some industrial use, e.g., NASA, Intel, ...

```
inBuf = (char*) malloc(bufSz);  
if (inBuf == NULL)  
    return -1;  
outBuf = (char*) malloc(bufSz);  
if (outBuf == NULL)  
    return -1;
```

Bug (pattern) finders

- Work with a fault model of typical programmer mistakes

```
1 Person person = aMap.get("bob");  
2 if (person != null) {  
3     person.updateAccessTime();  
4 }  
5 String name = person.getName();
```

```
1 String b = "bob";  
2 b.replace('b', 'p');  
3 if(b.equals("pop"))
```

Null-pointer dereference

`person.getName()`

Ignored return value

`replace` returns updated string

- Note overlap with other methods
- Findbugs is best example of such a tool

Why is code scanning hard?

Theory and practice

Why is it hard in theory?

- **Flaws** concern problematic behavior
 - **Failures**: deviation of behavior detectable at system interface
 - **Faults**: deviation of (internal) behavior from intended one
- Behavioral properties in general undecidable.
 - Termination, reachability of a program point,...
- Tools must therefore
 - if program p halts
call insecure();
 - **not always terminate**, or
 - **over-approximate** behavior (returning **false positives**), or
 - **under-approximate** behaviors (returning **false negatives**)

And in practice?

Coverity experience (“A few billion lines of code later”, CACM, 2010)

- Tool finds generic errors and interface-specific violations
E.g., memory corruption, data races, function-ordering constraints, ...

- Company’s goals: build user community, \$\$\$

- **Central religion is results**: good = works

- **Find errors**, not prove their absence
- **Embrace unsoundness!** Focus on low-hanging fruit



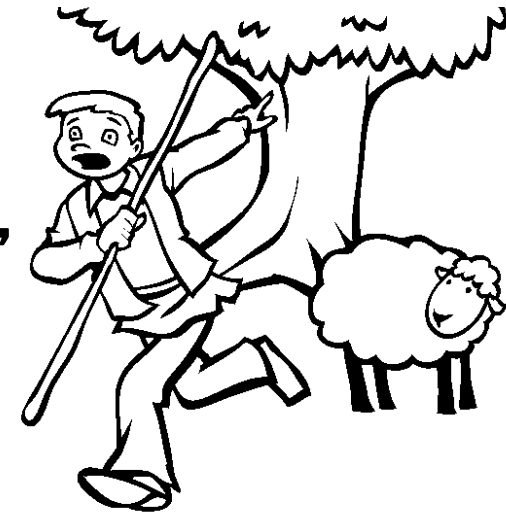
“Circa 2000, unsoundness was controversial in the research community, though it has since become almost a de facto tool bias for commercial products and many research projects”

- **Usability** and **simplicity** are critical. For both sales and deployment

Coverity experience

“Theory and practice are similar, but only in theory”

“A few billion lines of code later”, CACM, 2010



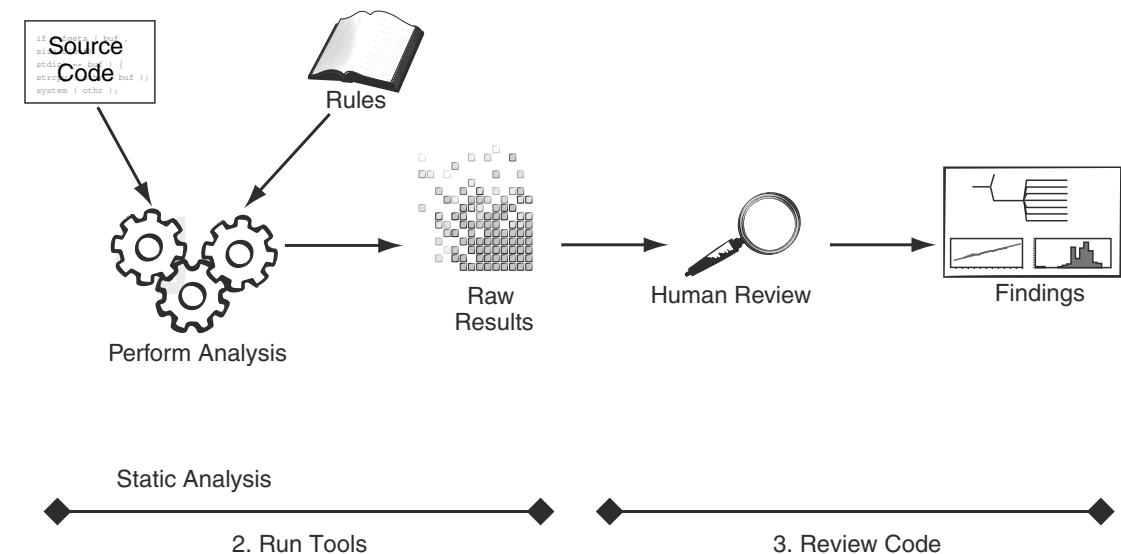
- Hardest part is to find the code
 - Home-brown build environments, special tools make problem hard
 - Solution: run build process and intercept all system calls
 - But system must now build!
- And you have to parse it
 - Experience: the C language doesn't exist. Same for C++, Java, ...
 - Various versions of compilers allow various code to appear in codebases
- Give it semantics (different PLs, external libraries, system calls, configuration)
- Customers must understand the bugs and care about them
 - Give informative output, rank severity and confidence, etc.
- No churn: tool output shouldn't change much, even after upgrades!
- Avoid false positives. More than 30% $\Rightarrow$ tool won't be used/sell.
 - In general, manage the precision-depth-scalability tradeoff

Basic dilemma: cry wolf vs. missing detail kills!

How are code scanners used?

Code review cycle

- Scanners used within a code review
Set goals, run tools, review code, fix it
- Often augmented by adding new rules to scanner
 - Rules encoding coding-style/company/product specific rules
 - Rules motivated by problems not found by scanner
- For fixes, developers must first be convinced. **Exploitability trap.**
 - Often result is not unambiguously secure or exploitable
 - Difficult to motivate improvements. Especially for style issues.
- **Code review should be more than scanning! Why?**



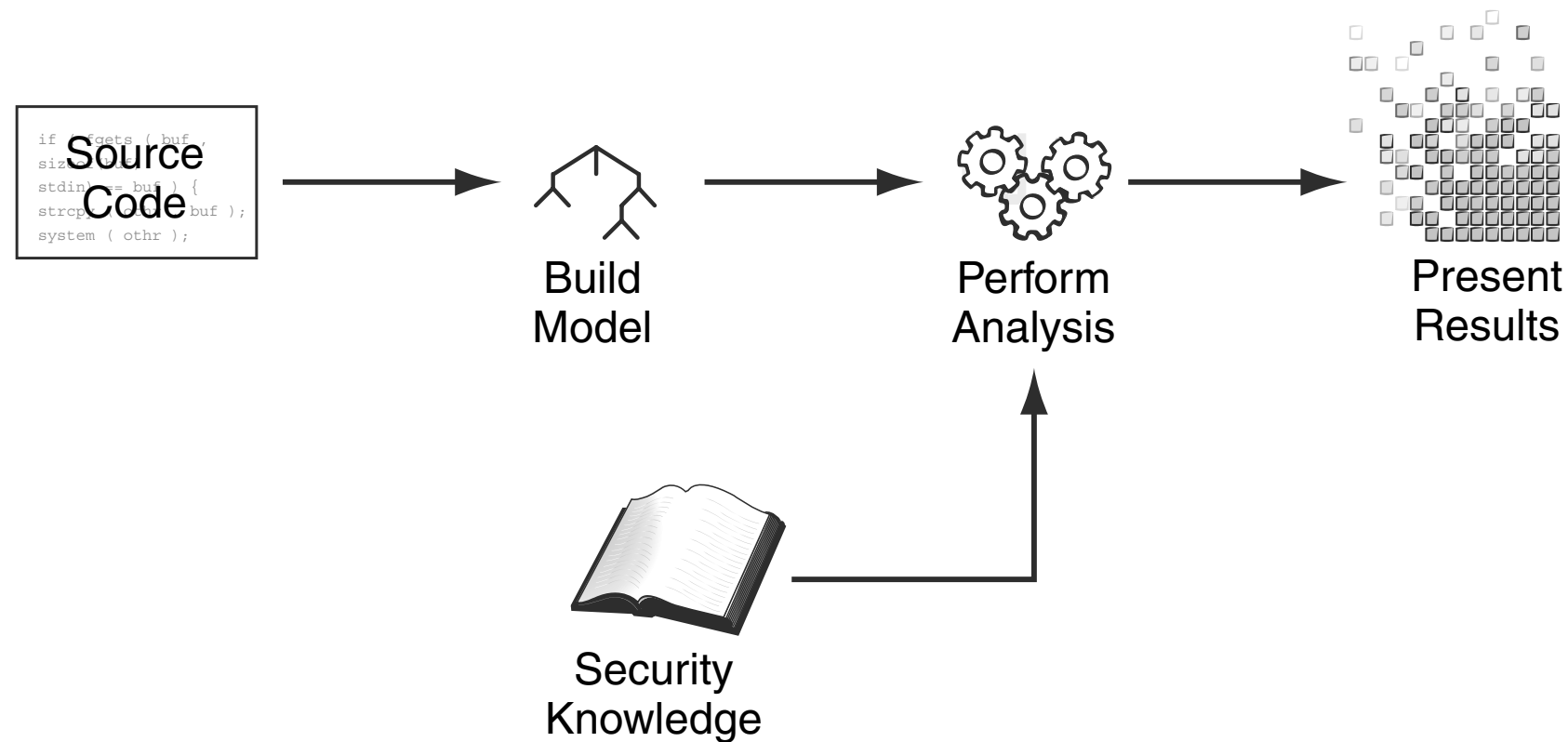
In projects, embed code scanning within code review!



Code scanning basics

Introduction to (parts of) the tool bag

Overview



- We use Fortify tool as rough “tool model” in the lectures
See Chess & West “*Secure Programming with Static Analysis*”, 2007
- You will use similar open-source tools in the exercises and the lab
- We provide an abstract account here
- Afterwards, examine specialized techniques for different bugs
- Focus on bugs that represent **security vulnerabilities**

Model-building — from source to AST

Standard from compiler design

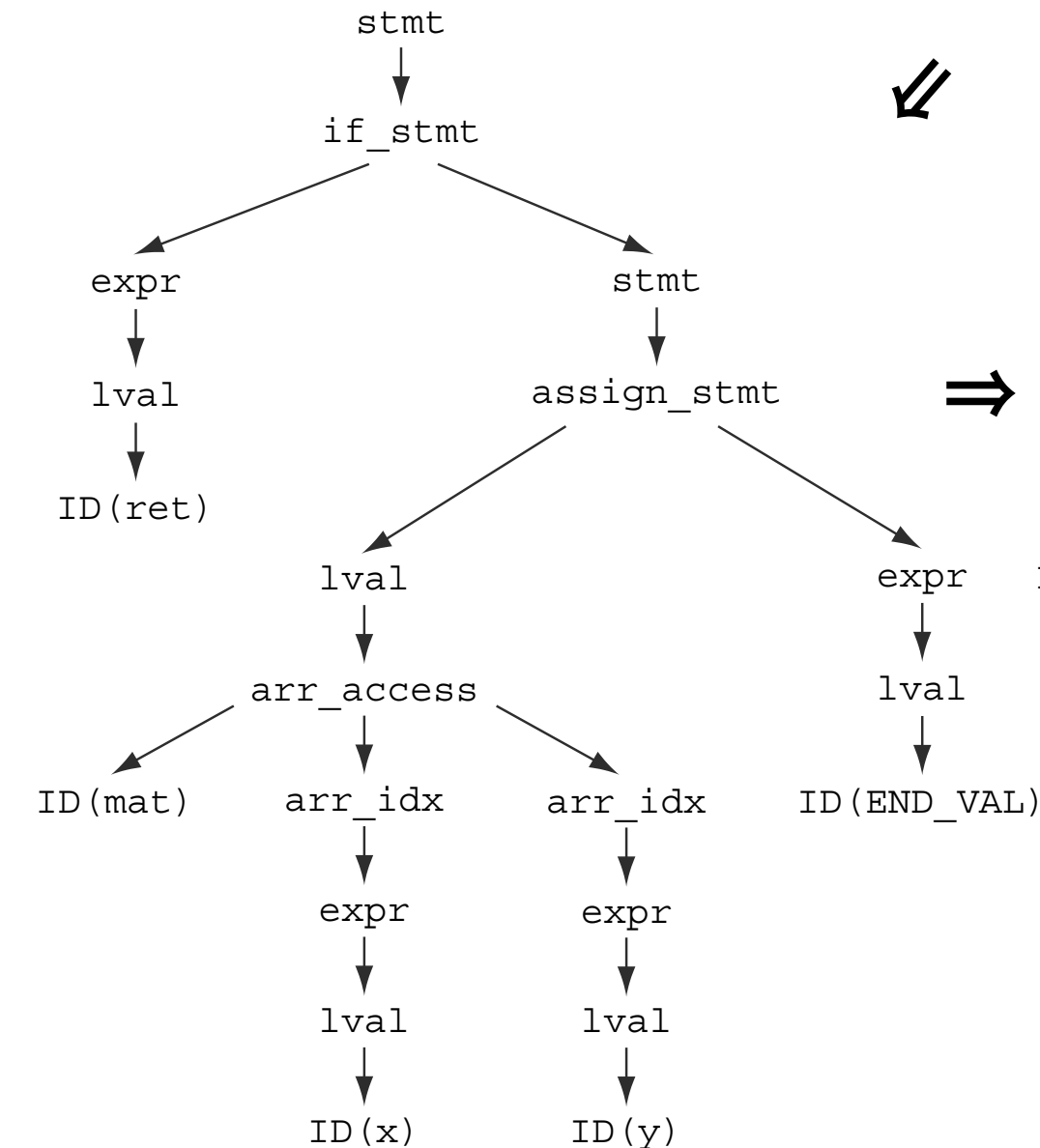
if (ret) // probably true
mat[x][y] = END_VAL;



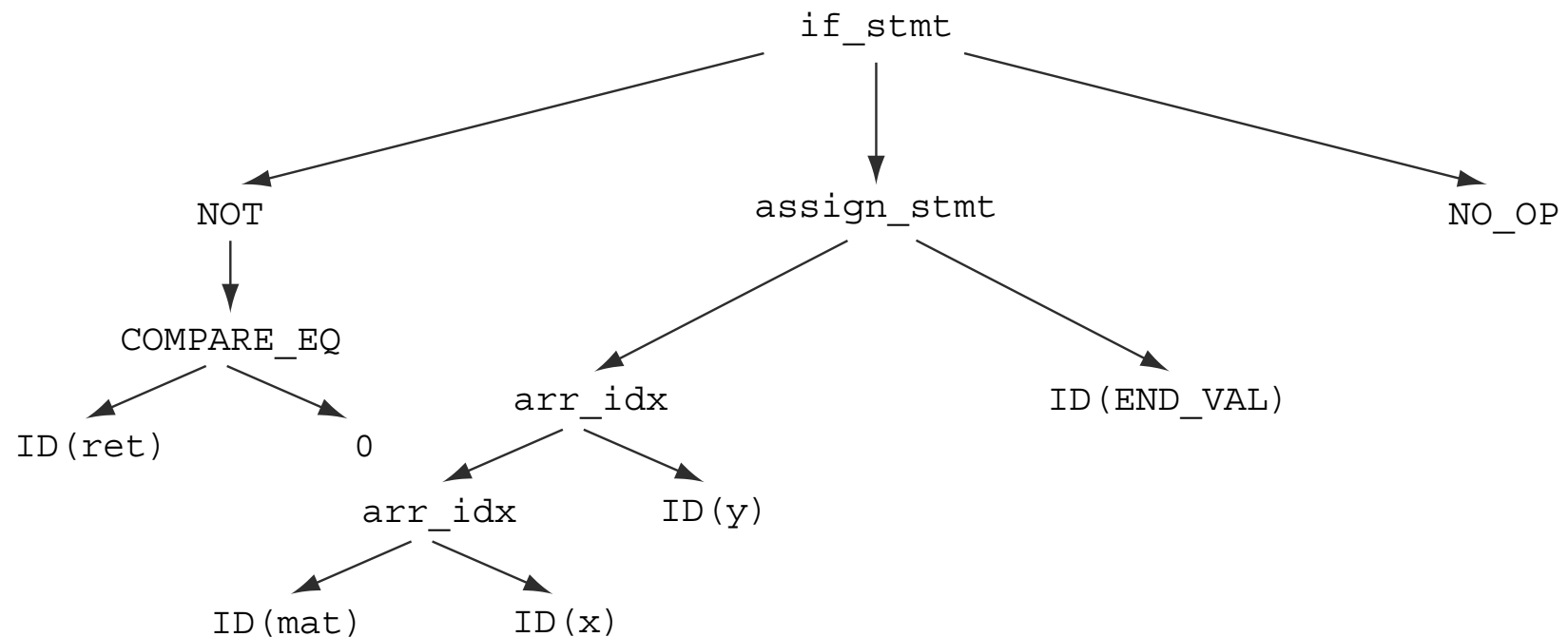
IF LPAREN ID(ret) RPAREN ID(mat) LBRACKET ID(x) RBRACKET LBRACKET
ID(y) RBRACKET EQUAL ID(END_VAL) SEMI

Source

Token stream



Parse tree



Abstract syntax tree

Abstracts away details of grammar and makes
language aspects explicit (e.g., if(p) is if(p neq 0)).

Can also de-sugar the language,
e.g., only one kind of loop, explicate type conversions.

Semantic analysis and structural rules (I)

ID	Type	Static	AST ptr
ret	int	no	...
mat	int*	no	...

- Build **symbol table** along with AST (as in compilers)
- This can be used for type checking, which helps find bugs
- Types also help in specifying “structural rules” for bug finding
- **Example:** Java DB connections should not be shared between threads
 - So DB connections should not be stored in static fields
 - To find instances of this problem look for

Field: `static` and `type.name` = “`java.sql.Connection`”

- Above is an example of a Fortify “structural rule”

Semantic analysis and structural rules (II)

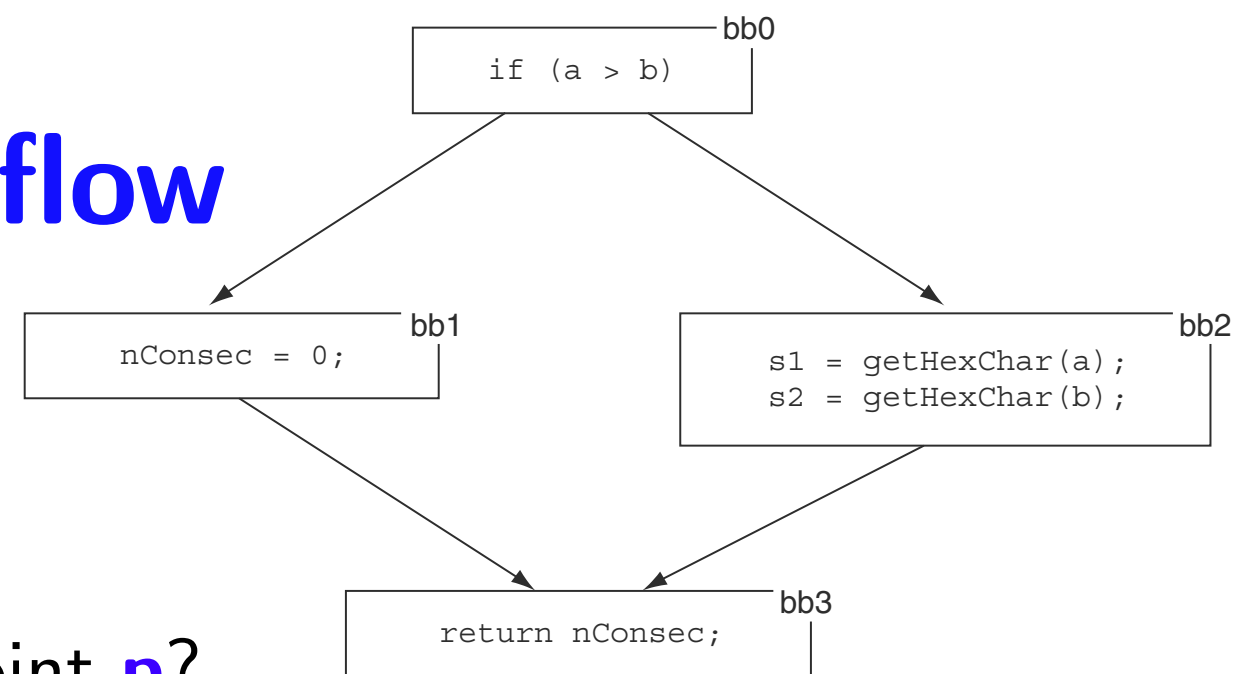
- **Example:** `buf = realloc(buf, 256)`
 - Memory leaks If `realloc()` fails and returns `null`
 - Can detect with following structural rule
FunctionCall c1: (
 c1.function is [name == "realloc"] and
 c1 in [AssignmentStatement: rhs is c1 and
 lhs == c1.arguments[0]])
- So some overlap between compilers and code scanners
 - Compiler type checking **is** code scanning
 - Example **lint**: finds suspicious issues and portability bugs for C

Going beyond...

- Previous examples were simple
 - Compare type with field kind
 - Compare lhs with argument of function call
- To go beyond this need to reason about
 - **control flow**: how instructions are sequenced
 - e.g., are acquired locks released?
 - **data flow**: how data moves through program variables
 - e.g., does input come from a network-facing interface?

Let's look at **basic approaches** to these problems

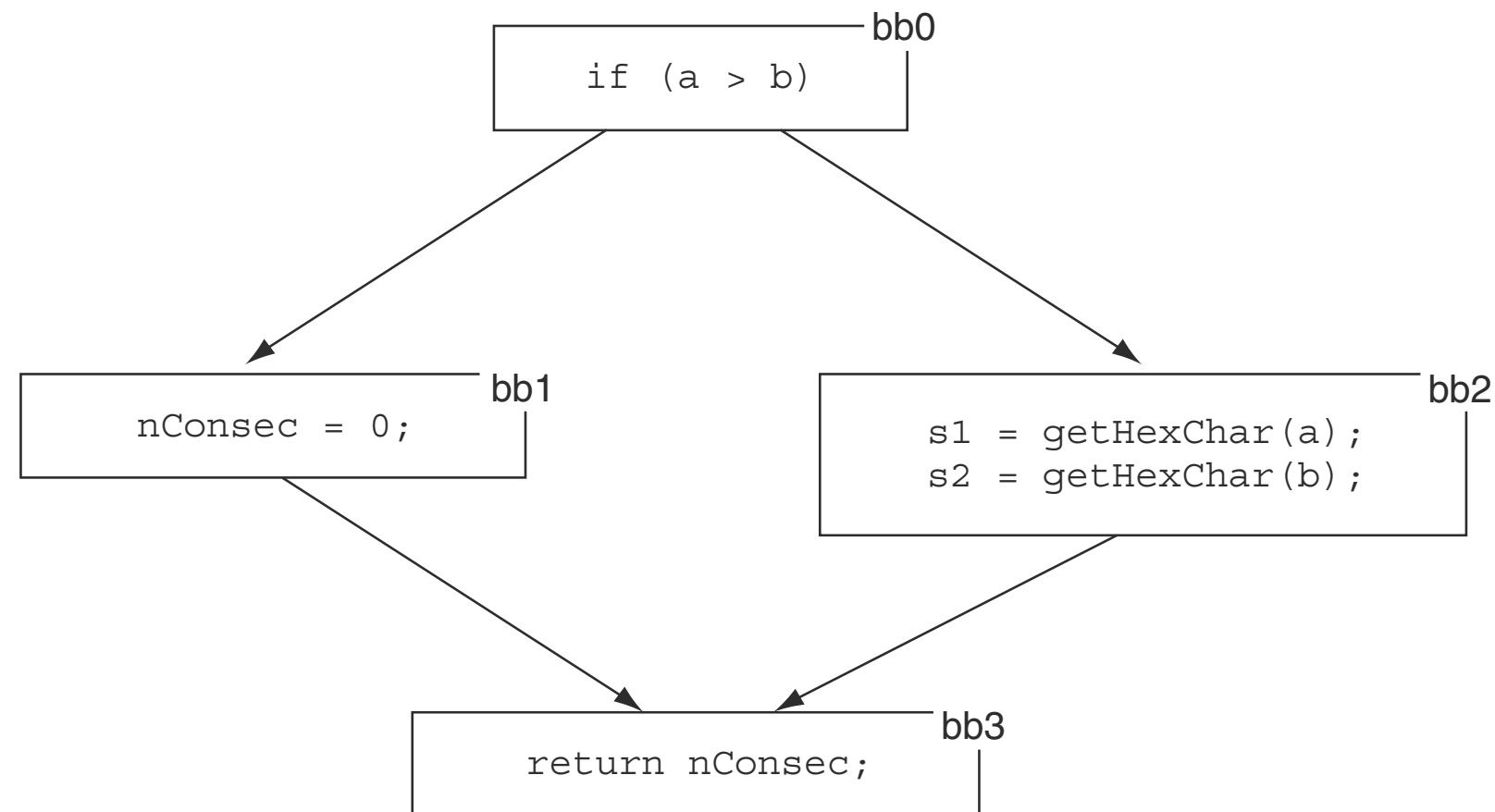
Control flow



- Need to answer questions like
 - Which execution paths lead to point **p**?
 - Which execution paths follow from **p**?
- Tools build **control flow graph** on top of AST
 - **Basic block**: sequence of instructions that is always executed, i.e., no jumps in/out of middle, no branching, etc.
 - **Forward edges**: potential control flow paths between BBs
 - **Backward edges**: possible loops
- System **traces** in terms of BB sequences

Control flow example

```
if (a > b) {  
    nConsec = 0;  
} else {  
    s1 = getHexChar(1);  
    s2 = getHexChar(2);  
}  
return nConsec;
```

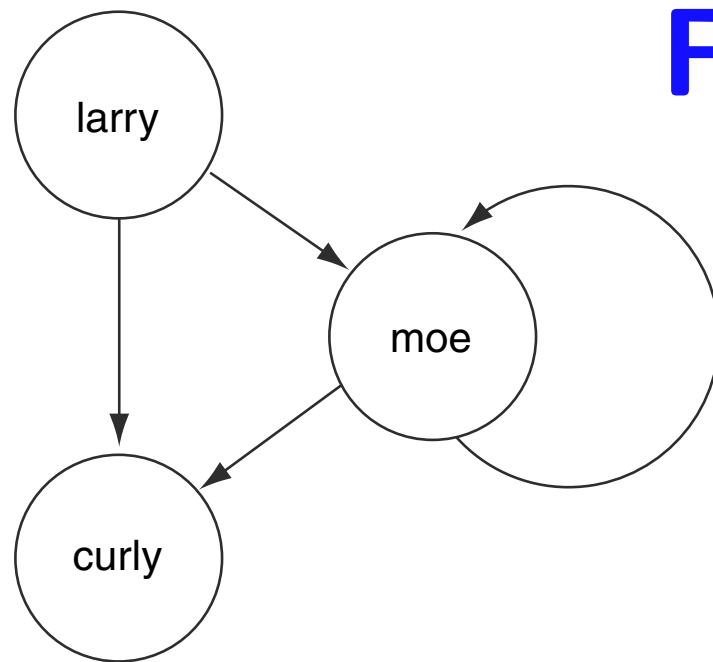


- In practice, graph nodes store pointers to AST nodes
- Two traces: [bb0, bb1, bb3] and [bb0, bb2, bb3]
- **Gretchenfrage** for bug finders:
report bugs in **some** versus **all** traces?

Nun sag, wie hast du's mit der Religion? Du bist ein herzlich guter Mann, allein ich glaub, du hältst nicht viel davon



Function call graph



- Represents control flow between functions and/or methods
- **Nodes** = functions; **Edges** = potential invocation
- Different representations may need to be combined
 - Traces will depend on function calls
 - Even trickier in practice, e.g., with function pointers or virtual methods must also use **dataflow** analysis and **datatype** analysis to limit the set of potential functions that can be invoked from a call site.

```
int larry(int fish) {  
    if (fish) {  
        moe(1);  
    } else {  
        curly();  
    }  
}  
  
int moe(int scissors) {  
    if (scissors) {  
        curly();  
        moe(0);  
    } else {  
        curly();  
    }  
}  
  
int curly() {  
    /* empty */  
}
```

Dataflow analysis

- Determine how data moves through program
 - Traverse control flow graph and note where data generated and used
- Implementation trick: convert function to **Static Single Assignment**
 - Can assign to a variable only once, so make unique with indices
 - Makes it trivial to determine where value comes from
- Constant propagation
 - If SSA variable assigned a constant, then replace it by constant
- **Question**: any security relevant applications of constant propagation?
Find hard-coded passwords or encryption keys!

Dataflow examples

<pre>sum = sum + delta ; sum = sum & top; y = y + (z<<4)+k[0] ^ z+sum ^ (z>>5)+k[1]; y = y & top; z = z + (y<<4)+k[2] ^ y+sum ^ (y>>5)+k[3]; z = z & top;</pre>	<pre>sum₂ = sum₁ + delta₁ ; sum₃ = sum₂ & top₁; y₂ = y₁ + (z₁<<4)+k[0]₁ ^ z₁+sum₃ ^ (z₁>>5)+k[1]₁; y₃ = y₂ & top₁; z₂ = z₁ + (y₃<<4)+k[2]₁ ^ y₃+sum₃ ^ (y₃>>5)+k[3]₁; z₃ = z₂ & top₁;</pre>
---	---

Example 1 (tiny encryption): simple, straight-line code

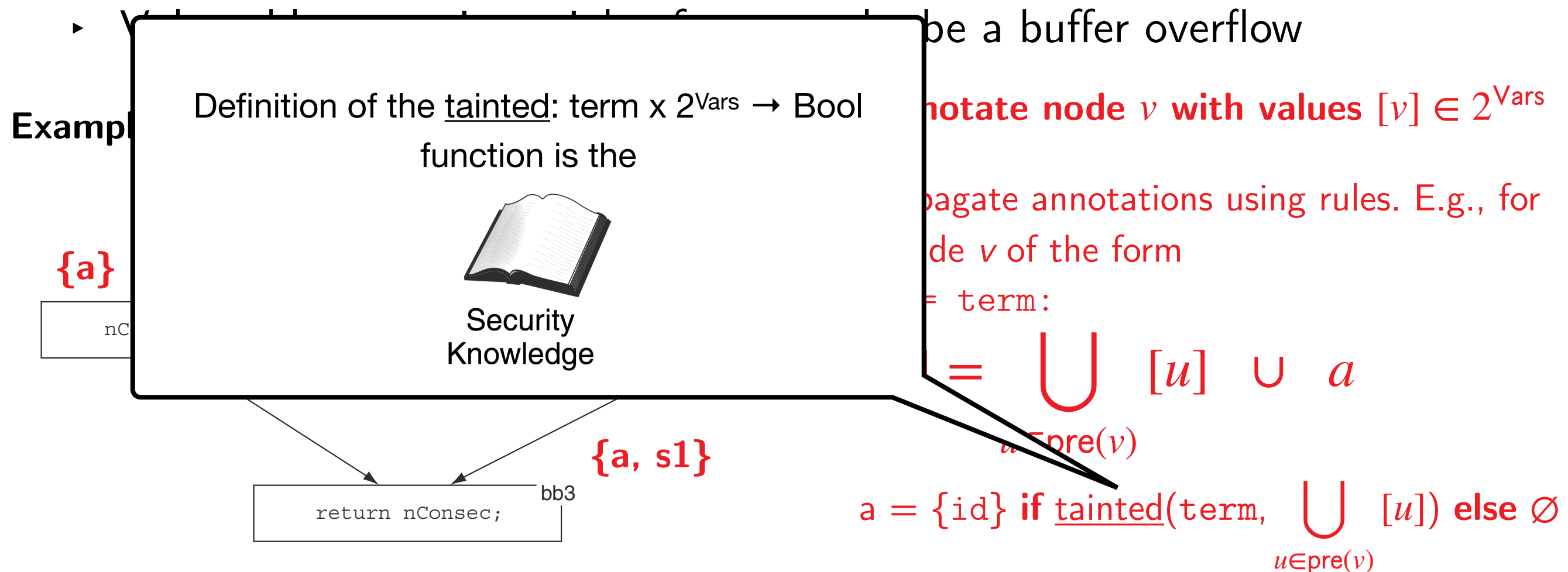
```
if (bytesRead < 8) {
    tail = (byte) bytesRead;
}
```

```
if (bytesRead1 < 8) {
    tail2 = (byte) bytesRead1;
}
tail3 =  $\phi$ (tail1, tail2);
```

Example 2 (branching): requires reconciling variable where control paths merge. Done by introducing phoney “selection function” ϕ .

Dataflow application: taint analysis

- What values can an attacker potentially control?
- Solve by tracing how the attacker's input moves through program
 - Existence of path from input function to a vulnerable operation



For loops: need to reach a fix-point!

- Taint analysis/tracking can also be done dynamically (e.g., Perl)



Analysis algorithms

Improving power and precision

Need for sophistication

- **Example:** strcpy(dest, src)
 - You should probably never use it. But not all occurrences are harmful
 - If **assertion** `alloc_size(dest) > strlen(src)` always succeeds, no overflow
- Analogous situation for: SQL injections, cross-site scripting,...
- So let's treat code scanning as a property-checking problem!
 - **Question 1:** how do you check properties?
 - **Question 2:** which properties do you check?
 - **Taint propagation:** is value tainted? (How do you formalize this?)
 - **Range analysis:** relevant, e.g., for buffer overflows
 - **Type state:** is variable in right “type” of state at given point?
E.g., free(p) called in state where p not already freed

Q1: How do you check properties?

Symbolic execution (simple case)

- Consider a simple program and its possible analysis

$x = 1;$	$\{\}$	(no facts)	
$y = 1;$	$\{x = 1\}$		
$\text{assert } (x < y)$	$\{x = 1, y = 1\}$		$\times$

- Conclude $1 < 1$, i.e., report that assertion always fails!
- Same conclusion holds if “1” is replaced with “v”

This is **symbolic execution** and conclusion holds for all v

- So straight-line code is easy! Where do the problems lie?

Q1: How do you check properties?

Symbolic execution (branches)

Program	Case 1: then branch	Case 2: else branch
$x = v;$	$\{x = x_0, y = y_0\}$	$\{x = x_0, y = y_0\}$
$\text{if } (x < y) \{$	$\{x = v, y = y_0\}$	$\{x = v, y = y_0\}$
$y = v; \}$	$\{x = v, y = y_0, x < y_0\}$	
$\text{assert } (x < y)$	$\{x = v, x < y_0, y = v\}$ ✗	$\{x = v, y = y_0, \neg(x < y_0)\}$ ✗

- Evaluate **assertion** with all the (normalized) information
 - Case 1: $x < y \wedge x = v \wedge x < y_0 \wedge y = v$. Implies $v < v$, ⚡
 - Case 2: $x < y \wedge x = v \wedge y = y_0 \wedge \neg(x < y_0)$. Implies $x < y \wedge \neg(x < y)$, ⚡So assertion always fails
- Problem: #paths is exponential in #conditions
 - Share common subpaths
 - Eliminate false paths
 - Problem slicing is one technique to cope with this (not covered here)
 - Loops even more problematic. Solutions? What about random numbers?

Q1: How do you check properties?

Assertion checking in practice

- Fully static assertion checking is deductive verification!
 - Long history of work on this, both interactive and automated
- Predicate transformers + theorem proving
 - For program S & predicate R , $wp(S,R)$ is weakest precondition on initial state such that S terminates in a final state satisfying R
 - R may be desired assertion
 - Theorem proving is needed in general

$$\begin{aligned} wp(\text{if } x < y \text{ then } x := y \text{ else skip end}, x \geq y) &= (x < y \Rightarrow wp(x := y, x \geq y)) \wedge (\neg(x < y) \Rightarrow wp(\text{skip}, x \geq y)) \\ &= (x < y \Rightarrow y \geq y) \wedge (\neg(x < y) \Rightarrow x \geq y) \\ &\Leftrightarrow \text{true} \end{aligned}$$

- Loops require inventing loop invariant (or approx. by finite unrolling)
- Model checking, but then state space should be finite

Q2: Which properties do you check?



- Research suggest that the **secret sauce** is the properties!
- Different options
 1. Let programmer annotate program with **assertions** or write **rules**
 2. Use standard set based on **fault models** or **bug categories**
 3. Infer rules based on “**consistency**”

If a method is called 100 times and return value is used 99 times, then flag non-checking instance as suspicious

Option 2 appears most popular in practice.

Example (external) rules

```
<Vulnerability>
  <Name>system</Name>
  <InputProblem>
    <Arg>1</Arg>
    <Severity>High</Severity>
  </InputProblem>
</Vulnerability>
```

```
<DataflowSinkRule formatVersion="3.2" language="cpp">
  <MetaInfo><Group name="package">C Core</Group></MetaInfo>
  <RuleID>AA212456-92CD-48E0-A5D5-E74CC26A276F</RuleID>
  <VulnKingdom>Input Validation and Representation</VulnKingdom>
  <VulnCategory>Command Injection</VulnCategory>
  <DefaultSeverity>4.0</DefaultSeverity>
  <Description ref="desc.dataflow.cpp.command_injection"/>
  <Sink>
    <InArguments>0</InArguments>
    <Conditional>
      <Not>
        <TaintFlagSet taintFlag="VALIDATED_COMMAND_INJECTION"/>
      </Not>
    </Conditional>
  </Sink>
  <FunctionIdentifier>
    <FunctionName><Value>system</Value></FunctionName>
  </FunctionIdentifier>
</DataflowSinkRule>
```

- **RATS rule** (lhs): reports violation whenever 1st argument to **system()** is not a constant (RATS = Rough Auditing Tool for Security)
- **Fortify rule** (rhs): reports same violation but only if there is a path through program where attacker can control the 1st argument, which has not been validated to prevent injections
- ◆ Also classifies vulnerability, links to textual description, etc.

Making rules precise — taint analysis

```
    ①  
if ( fgets ( buf , sizeof(buf), stdin) == buf ) {  
    ②  
    ③  
    strcpy ( othr , buf );  
    ④  
    ⑤  
    system ( othr );  
}
```

The diagram illustrates a code snippet with five numbered annotations. Annotation 1 points to the `buf` parameter in the `fgets` function call. Annotation 2 points to the `buf` parameter in the `strcpy` function call. Annotation 3 points to the `othr` parameter in the `strcpy` function call. Annotation 4 points to the `othr` parameter in the `system` function call. Annotation 5 points to the `system` function call. Arrows indicate the flow of data from `buf` in `fgets` to `buf` in `strcpy`, and from `othr` in `strcpy` to `othr` in `system`.

- ① A source rule for `fgets()` taints `buf`
- ② Dataflow analysis connects uses of `buf`
- ③ A pass-through rule for `strcpy` taints `othr`
- ④ Dataflow analysis connects uses of `othr`
- ⑤ Because `othr` is tainted, a sink rule for `system()` reports a command injection vulnerability

- **Source rules** define locations where tainted data enters system
- **Sink rules** define locations that should not receive tainted data
- **Pass-through rules** define how a function manipulates tainted data
E.g., if input is tainted, output is similarly (or differently) tainted
- **Cleanse rule**: special pass-through rules that remove taint
E.g., validation functions

Taint analysis rules (cont.)

- Taint is just an attribute on data. In practice not binary
 - Recall the binary 2^{Vars} annotations
 - Input from **main()** may not be trustworthy, but guaranteed to be null terminated string
 - Data may have undergone certain checks
 - So represent taint by **different taint flags**
- Sinks may be dangerous only when data has a certain taint
- Rules can manipulate taint in either **additive** or **subtractive** manner
- All of this must be specified, also for user-defined functions
 - Alternative: resort to binary taint, but with many false positives
 - Engineering effort is most of the work



Handling Input

Why, what, and how?

Input validation — overview

- Security maxim: **Validate all input!**
- **Attack surface** = all places where program accepts input
 - All program entry points and function calls taking external input
 - Can use static analysis tools to help audit these!
- Attack surface includes:
Command-line parameters, configuration files, database data, environment variables, network services, registry values, temp. files
- All of these have been exploited in the past!
(See Chess/West for dozens of examples)
- We will look at several examples and how scanning helps

Example: Database Access

```
ResultSet rs = stmt.executeQuery();  
rs.next();  
int balance = rs.getInt(1);
```

- The database is input too! Is above code OK?
 - Next positions cursor to first row
 - `rs.getInt(1)` gets value of first column, here the balance
- What if database is corrupted? Security/Functionally relevant!
 - Could be zero or multiple rows. This should be checked.
- Fix: validate input!

```
ResultSet rs = stmt.executeQuery();  
if (!rs.next()) {  
    throw new LookupException("no balance row");  
}  
if (!rs.isLast()) {  
    throw new LookupException("more than one balance row");  
}  
int balance = rs.getInt(1);
```

Rule to find this problem

- Identify where program ignores number of rows in a ResultSet
 - Find calls to `ResultSet.next()` that are call statements and therefore not in a predicate

```
FunctionCall fc:  
  (fc.function is [name == "next" and  
                  enclosingClass.supers contains  
                  [Class: name == "java.sql.ResultSet"]]) and  
  (fc in [CallStatement:])
```



- **Question:** why wouldn't you trust your data base?
 - Don't trust anything! Stop compromise escalation!
 - But if almost everything is tainted, analysis should be precise



Conclusions

Hundreds of properties and rules

- Chess/West book is an excellent introduction to them
 - Primer on secure programming and security functionality
 - E.g., stack/heap overflows, safe language dialects, error handling, input validation frameworks, XML problems, safe storage of passwords, race conditions, ...
 - Highly recommended reading!
- Name of the game stays the same: vulnerability patterns + rules
- Experience of different tool makers similar
 - Simple analysis combined with good rules yields dramatic results

But how simple is the analysis?

Findbugs experience

simple analysis combined with effective rules

Implementation strategies used by the detectors can be roughly categorized into:

- **Class structure and inheritance hierarchy only.** Some of the detectors simply look at the structure of the analyzed classes without looking at the code.
- **Linear code scan.** These detectors make a linear scan through the bytecode for the methods of analyzed classes, using the visited instructions to drive a state machine. These detectors do not make use of complete control flow information; however, heuristics (such as identifying the targets of branch instructions) can be effective in approximating control flow.
- **Control sensitive.** These detectors use an accurate control flow graph for analyzed methods.
- **Dataflow.** The most sophisticated detectors use analysis that takes both control and data flow into account. An example is the null pointer dereference detector.

None of the detectors make use of analysis techniques more sophisticated than what might be taught in an under-graduate compiler course.

Simple analysis ≠ simple rules or concepts

Double-Checked Locking (DC) rule

- Consider “lazy initialization”, delaying initialization of owned object until it is really needed

```
class SomeClass {  
    private Resource resource = null;  
    public Resource getResource() {  
        if (resource == null)  
            resource = new Resource();  
        return resource;  
    }  
}
```

OPTION 1: test and set

```
class SomeClass {  
    private Resource resource = null;  
    public Resource getResource() {  
        if (resource == null) {  
            synchronized {  
                if (resource == null)  
                    resource = new Resource();  
            }  
        }  
        return resource;  
    }  
}
```

DCL variant for multi-threading

- Option 1 has a race condition
- Option 2: compilers, microprocessors, and caches can do reordering
 - It is possible thread A enters `synchronized` and calls `new Resource()`
 - Thread B enters `getResource()`
 - Due to reordering, B gets a non-null reference to a partially built resource!
- Its very subtle! Read the “Double-Checked Locking is Broken” Declaration
- As of JDK1.5 the problem is resolved by making the resource field **volatile**

Evaluation (partial)

	code	classpath-0.08				rt.jar 1.5.0 build 59			
		warnings	serious	mostly harmless	false pos	warnings	serious	mostly harmless	false pos
☛	DC	1	100%	0%	0%	88	100%	0%	0%
	EC	0	—	—	—	8	100%	0%	0%
	IS2	46	58%	23%	17%	116	44%	47%	7%
	NP	7	85%	0%	14%	37	100%	0%	0%
	NS	0	—	—	—	12	25%	66%	8%
	OS	6	50%	0%	50%	13	15%	0%	84%
	RCN	5	80%	0%	20%	35	57%	0%	42%
	RR	9	100%	0%	0%	12	91%	0%	8%
	RV	5	100%	0%	0%	7	71%	0%	28%
	UR	3	66%	0%	33%	4	100%	0%	0%
	UW	2	0%	0%	100%	6	50%	0%	50%
	Wa	3	0%	0%	100%	8	37%	0%	62%

	code	eclipse-3.0				drjava-stable-20040326			
		warnings	serious	mostly harmless	false pos	warnings	serious	mostly harmless	false pos
☛	DC	88	100%	0%	0%	0	—	—	—
	EC	19	57%	0%	42%	0	—	—	—
	IS2	63	61%	22%	15%	2	0%	0%	100%
	NP	70	78%	7%	14%	0	—	—	—
	NS	14	78%	21%	0%	0	—	—	—
	OS	26	46%	0%	53%	4	100%	0%	0%
	RCN	69	40%	11%	47%	0	—	—	—
	RR	39	38%	0%	61%	0	—	—	—
	RV	8	100%	0%	0%	0	—	—	—
	UR	4	50%	50%	0%	1	0%	100%	0%
	UW	7	28%	0%	71%	3	100%	0%	0%
	Wa	12	25%	0%	75%	3	100%	0%	0%

Summary

- Code scanning should play a central role in code review
 - Aids understanding code
 - Helps finding common bugs
- Surprisingly effective!
 - Everyone makes dumb mistakes
 - Simple patterns can describe remarkably subtle bugs
 - Threads/crypto/... more complex than most people think
- Pragmatic conservative static analysis techniques play a major role
 - Augmented with lots of domain knowledge
- Topic is deep. We only scratched surface on analysis methods

Literature

- Chess & West “Secure Programming with Static Analysis”, 2007
- “Mars Code”, CACM Feb. 2014
- “A few billion lines of code later”, CACM, 2010
- Martin & Livshits & Lam “Finding application errors and security flaws using PQL: a program query language”, OOPSLA 2005
- Hovemeyer & Pugh “Finding Bugs is Easy”, OOPSLA 2004
- Tsipenyuk, Katrina, Brian Chess, and Gary McGraw. "Seven pernicious kingdoms: A taxonomy of software security errors." *IEEE Security & Privacy* 2005